

08. Contact Tickets Deep Dive

1. Trang contact-tickets giải quyết bài toán gì?

Trang **contact-tickets** không chỉ là một form liên hệ. Nó là một hệ thống hỗ trợ theo ticket, nơi người dùng có thể:

- Mở một phiếu hỗ trợ mới bằng tài khoản đang đăng nhập.
- Theo dõi trạng thái xử lý của từng phiếu.
- Chat qua lại với thủ thư trong đúng ngữ cảnh của phiếu đó.
- Yêu cầu mở lại nếu chưa hài lòng.
- Đánh giá chất lượng hỗ trợ sau khi phiếu được xử lý.

Ở phía thư viện, thủ thư có một inbox riêng để:

- Xem toàn bộ phiếu hỗ trợ.
- Lọc theo trạng thái.
- Tìm kiếm theo mã phiếu, người gửi, email, tiêu đề, nội dung.
- Nhận xử lý một phiếu.
- Trả lời người dùng.
- Ghi chú nội bộ.
- Đánh dấu đã xử lý hoặc đóng phiếu.

Admin cũng có thể xem để giám sát, nhưng không được nhảy vào xử lý. Đây là điểm rất quan trọng: hệ thống tách rõ quyền xem, quyền nhận xử lý và quyền cập nhật.

2. Những bên đã nhảy vào hệ thống này

Một trang tốt như **contact-tickets** không đến từ một component đơn lẻ. Nó là kết quả của nhiều lớp phối hợp:

Bên tham gia	Vai trò
User	Người mở phiếu, bổ sung thông tin, chat với thư viện, đánh giá hoặc yêu cầu mở lại
Librarian	Người nhận xử lý, phản hồi, ghi chú nội bộ, resolve/close ticket
Admin	Người giám sát read-only, xem nội dung và đánh giá chất lượng
Frontend user page	LMS_FE/pages/user_pages/ContactTicketsPage.tsx
Frontend librarian/admin page	LMS_FE/pages/librarian_pages/ContactInbox.tsx
API client	LMS_FE/api/contactService.ts
Backend controller	LMS_BE/library-user-module/.../ContactMessageController.java

Bên tham gia	Vai trò
Database	<code>contact_messages</code> , <code>contact_message_comments</code> , <code>contact_message_internal_notes</code>
Notification system	Kafka notification khi ticket có reply/resolved/closed
Email system	Email khi ticket được resolved

Vì vậy, gọi đây là "trang liên hệ" thì hơi nhẹ. Đúng hơn, đây là một mini helpdesk system nằm trong LMS.

3. Tư duy cốt lõi: ticket là aggregate gốc

Đối tượng trung tâm là `contact_messages`. Mỗi dòng trong bảng này đại diện cho một ticket.

Các thông tin chính:

- `ticket_code`: mã phiếu dạng `REQ-XXXXXXX`, dùng cho người dùng và thủ thư nói chuyện với nhau dễ hơn.
- `sender_user_id`, `sender_name`, `sender_email`: định danh người gửi.
- `category`: nhóm hỗ trợ như lỗi hệ thống, mượn/trả, đề xuất sách, tài khoản.
- `subject`, `message`: tiêu đề và nội dung ban đầu.
- `status`: vòng đời của ticket.
- `assigned_to_user_id`: thủ thư đang phụ trách.
- `handled_by_user_id`: người đã thao tác xử lý gần nhất.
- `reply_message`, `replied_at`: phản hồi gần nhất, phục vụ hiển thị nhanh.
- `closed_at`, `reopened_at`: mốc thời gian đóng/mở lại.
- `satisfaction_rating`, `feedback_note`: đánh giá của người dùng.

Ticket là aggregate gốc vì mọi thứ khác bám vào nó:

- Chat thread nằm trong `contact_message_comments`.
- Ghi chú nội bộ nằm trong `contact_message_internal_notes`.
- Notification/email đều dùng ticket làm reference.

Tư duy này giúp hệ thống rõ ràng: muốn biết một yêu cầu hỗ trợ đang ở đâu, nhìn vào `contact_messages`; muốn biết cuộc trao đổi chi tiết, đọc comments; muốn biết trao đổi nội bộ, đọc internal notes.

4. Vòng đời trạng thái

Ticket hiện có 4 trạng thái:

Trạng thái	Ý nghĩa
<code>NEW</code>	Người dùng vừa gửi, chưa có thủ thư nhận xử lý
<code>IN_PROGRESS</code>	Đã có thủ thư nhận hoặc có trao đổi đang diễn ra
<code>RESOLVED</code>	Thủ thư đã xử lý xong, user có thể đánh giá hoặc yêu cầu mở lại
<code>CLOSED</code>	Phiếu đã đóng, không còn cần trao đổi

Luồng cơ bản:

```

USER tạo ticket
-> NEW

LIBRARIAN nhận xử lý
-> IN_PROGRESS

LIBRARIAN phản hồi
-> IN_PROGRESS

LIBRARIAN đánh dấu xử lý xong
-> RESOLVED

USER chưa hài lòng, yêu cầu mở lại
-> IN_PROGRESS

LIBRARIAN đóng phiếu
-> CLOSED

```

Điểm hay là **RESOLVED** không phải là "chấm hết". Nó là trạng thái đã xử lý nhưng vẫn để user phản hồi chất lượng hoặc yêu cầu mở lại. Đây là tư duy dịch vụ tốt hơn so với việc thủ thư bấm đóng ngay.

5. Làm sao để chat được với nhau?

Chat không dùng WebSocket. Hệ thống dùng mô hình thread theo ticket qua REST API.

Bảng **contact_message_comments** lưu từng tin nhắn:

- **contact_message_id**: thuộc ticket nào.
- **author_user_id**: user hoặc thủ thư nào gửi.
- **author_name**, **author_email**: snapshot thông tin người gửi tại thời điểm gửi.
- **author_role**: **USER**, **LIBRARIAN**, hoặc **SYSTEM**.
- **body**: nội dung tin nhắn.
- **created_at**: thời điểm gửi.

Khi user tạo ticket, backend không chỉ insert vào **contact_messages**, mà còn insert luôn comment đầu tiên vào **contact_message_comments**. Nhờ vậy giao diện conversation luôn có một thread thống nhất, không phải xử lý riêng "message ban đầu" và "message trả lời".

Luồng gửi tin:

```

Frontend gọi POST /contact-messages/{id}/comments
-> Backend kiểm tra quyền truy cập ticket
-> Backend kiểm tra ticket còn được phép chat không
-> Insert vào contact_message_comments
-> Update updated_at của contact_messages
-> Nếu người gửi là thủ thư: cập nhật reply_message, replied_at,
handled_by_user_id

```

-> Nếu cần: gửi notification cho user
-> Frontend append comment vào conversation

Phía user:

- Trang `ContactTicketsPage.tsx` gọi `contactService.myTickets()`.
- Khi chọn một ticket, gọi `contactService.comments(id)`.
- Khi gửi reply, gọi `contactService.addComment(id, body)`.

Phía thủ thư:

- Trang `ContactInbox.tsx` gọi `contactService.list({ limit: 200 })`.
- Khi chọn ticket, gọi song song:
 - `contactService.comments(id)`
 - `contactService.internalNotes(id)`
- Khi gửi tin, gọi `contactService.addComment(id, body)`.

Điểm thực dụng: đây là "chat theo lượt" chứ không phải realtime push. Người dùng bấm gửi là thấy tin nhắn của mình ngay, thủ thư bấm refresh hoặc chọn lại ticket là thấy cập nhật. Với bài toán thư viện, cách này đủ ổn, đơn giản hơn WebSocket, ít lỗi production hơn.

6. Critical section nằm ở đâu?

Critical section quan trọng nhất là thao tác "nhận xử lý ticket".

Bài toán: nếu hai thủ thư cùng mở inbox và cùng bấm "Nhận xử lý" trên một ticket **NEW**, hệ thống không được để cả hai cùng trở thành người phụ trách.

Nếu chỉ check ở frontend thì sẽ sai, vì hai browser có thể cùng nhìn thấy ticket chưa ai nhận. Nếu chỉ check bằng một câu SELECT rồi sau đó UPDATE riêng biệt thì vẫn có race condition:

```
Thủ thư A SELECT thấy assigned_to_user_id IS NULL
Thủ thư B SELECT thấy assigned_to_user_id IS NULL
Thủ thư A UPDATE thành A
Thủ thư B UPDATE thành B
```

Kết quả: người xử lý bị ghi đè.

7. Hệ thống giải quyết critical section như thế nào?

Backend xử lý ở endpoint:

```
POST /api/v1/contact-messages/{id}/assign
```

Logic chính:

```
UPDATE contact_messages
SET status = CASE WHEN status = 'NEW' THEN 'IN_PROGRESS' ELSE status END,
    assigned_to_user_id = :userId,
    handled_by_user_id = :userId,
    updated_at = CURRENT_TIMESTAMP
WHERE id = :id
    AND (assigned_to_user_id IS NULL OR assigned_to_user_id = :userId)
    AND status NOT IN ('RESOLVED', 'CLOSED')
RETURNING contact_messages.*;
```

Điểm cốt lõi nằm ở điều kiện **WHERE**:

```
assigned_to_user_id IS NULL OR assigned_to_user_id = :userId
```

Ý nghĩa:

- Nếu ticket chưa ai nhận, thủ thư hiện tại được nhận.
- Nếu chính thủ thư đó đã nhận rồi, thao tác idempotent, bấm lại không phá dữ liệu.
- Nếu ticket đã thuộc thủ thư khác, UPDATE không còn match.

Đây là cách đưa critical section xuống database, nơi có tính atomic của câu lệnh **UPDATE**.

Frontend chỉ đóng vai trò hỗ trợ UX:

- Nếu ticket đã có **assignedToUserId** khác current user, hiển thị **Chỉ xem – thủ thư khác đang xử lý**.
- Disable nút gửi tin, resolve, close.
- Chỉ cho thao tác khi **assignedToCurrentUser = true**.

Nhưng frontend không phải lớp bảo vệ cuối cùng. Lớp quyết định là backend + database.

8. Vì sao không để nhiều thủ thư cùng xử lý?

Với contact ticket, "một ticket - một owner" là lựa chọn tốt vì:

- Tránh hai thủ thư trả lời trái ngược nhau.
- Tránh người dùng nhận nhiều hướng dẫn khác nhau.
- Tránh trạng thái bị cập nhật đè.
- Giúp truy vết trách nhiệm rõ ràng.
- Giúp admin biết ai đang phụ trách.

Nếu muốn nhiều thủ thư cùng cộng tác, hệ thống vẫn có **internal_notes**. Đây là kênh để thủ thư ghi chú, bàn giao, lưu context mà user không nhìn thấy. Nhưng quyền reply/resolve vẫn nên thuộc owner để giữ nhất quán.

9. Phân quyền được thiết kế như thế nào?

Backend dùng các annotation:

- `@RequiresAuthentication`: bắt buộc đăng nhập.
- `@RequiresRole(RoleConstants.LIBRARIAN)`: chỉ thủ thư.
- `@RequiresAnyRole({RoleConstants.LIBRARIAN, RoleConstants.ADMIN})`: thủ thư hoặc admin.

Các rule chính:

Hành động	User	Librarian	Admin
Tạo ticket	Có	Không phải luồng chính	Không phải luồng chính
Xem ticket của mình	Có	Không áp dụng	Không áp dụng
Xem toàn bộ inbox	Không	Có	Có
Nhận xử lý	Không	Có	Không
Reply user	Có, trong ticket của mình	Có, nếu là thủ thư được assign	Không
Internal note	Không	Có	Xem được
Resolve/close	Không	Có, nếu được assign	Không
Reopen	Có, nếu ticket resolved/closed	Không	Không
Feedback	Có, nếu ticket resolved/closed	Không	Không

Đặc biệt, admin bị giới hạn read-only. Đây là thiết kế tốt vì admin giám sát chất lượng, còn thủ thư vận hành xử lý. Nếu admin cũng được xử lý tự do, audit trách nhiệm sẽ mờ.

10. Access control của user

User không được truyền email tùy ý để xem ticket người khác.

Khi tạo ticket:

- Backend lấy user hiện tại từ token.
- `sender_user_id`, `sender_name`, `sender_email` được lấy từ DB.
- Form frontend chỉ cho nhập category, subject, message.

Khi xem ticket:

```
WHERE cm.sender_user_id = :userId
OR LOWER(cm.sender_email) = LOWER(:email)
```

Điều này vừa hỗ trợ dữ liệu cũ dựa trên email, vừa ưu tiên định danh mới bằng `sender_user_id`.

Khi gọi `requireTicketAccess(id)`:

- Staff được xem.
- User thường chỉ được xem nếu email ticket khớp email tài khoản.
- Nếu không khớp, trả `403 FORBIDDEN`.

11. Ghi chú nội bộ khác chat như thế nào?

Chat nằm trong `contact_message_comments`, user nhìn thấy.

Ghi chú nội bộ nằm trong `contact_message_internal_notes`, chỉ staff/admin nhìn thấy.

Internal note dùng cho:

- Ghi lại hướng xử lý.
- Ghi lại thông tin nội bộ không nên gửi cho user.
- Bàn giao giữa thủ thư.
- Ghi chú lý do vì sao chọn resolve/close.

Rule bảo vệ:

- Chỉ librarian được thêm note.
- Chỉ tác giả note được sửa hoặc xóa note của mình.
- Admin xem được note nhưng không sửa/xóa.

Đây là một thiết kế rất quan trọng cho vận hành thật: người dùng thấy phần giao tiếp chính thức; nhân sự thư viện có thêm lớp ghi nhớ nội bộ.

12. Notification và email

Khi thủ thư phản hồi, resolve hoặc close ticket, backend gửi notification qua Kafka:

```
kafkaTemplate.send(KafkaTopics.NOTIFICATION_SEND, new  
NotificationMessage(...))
```

Các loại notification:

- `CONTACT_TICKET_REPLY`
- `CONTACT_TICKET_RESOLVED`
- `CONTACT_TICKET_CLOSED`

Link notification trở về:

```
/userpage/contact-tickets
```

Khi ticket được đánh dấu `RESOLVED`, backend còn gửi email bằng template `CONTACT_RESOLVED`.

Tư duy ở đây:

- Chat thread là nơi lưu sự thật.
- Notification là tín hiệu kéo user quay lại.
- Email là kênh nhắc ngoài hệ thống, nhất là khi user không mở web.

13. Vì sao UI của trang này tốt?

Trang tốt vì nó không chỉ "hiển thị dữ liệu", mà thiết kế theo workflow thật.

Phía user:

- Có dashboard nhỏ: đang mở, đã xử lý, tài khoản gửi.
- Có danh sách ticket bên trái, nội dung chi tiết bên phải.
- Có form tạo phiếu mới trong cùng màn hình.
- Khi ticket resolved/closed, ô chat bị khóa và thay bằng đánh giá hoặc yêu cầu mở lại.
- User không phải nhập lại tên/email vì hệ thống lấy từ tài khoản.

Phía thủ thư:

- Có summary cards theo trạng thái.
- Click card trạng thái là lọc inbox.
- Có search theo nhiều trường.
- Ticket list hiển thị status, category, sender, email, ticket code, thời gian.
- Chi tiết ticket hiển thị cả người gửi và thủ thư phụ trách.
- Có nút nhận xử lý rõ ràng.
- Nếu ticket thuộc thủ thư khác, UI chuyển thành read-only.
- Có mẫu phản hồi nhanh theo category.
- Có internal notes riêng.
- Có rating từ user để đánh giá chất lượng phục vụ.

Điểm mạnh nhất là UI không bắt thủ thư nhớ quy trình. Quy trình được encode thành trạng thái, nút bấm, disabled state và cảnh báo.

14. Response templates giúp gì?

Trong `ContactInbox.tsx`, hệ thống có nhiều mẫu phản hồi:

- Đã tiếp nhận.
- Cần bổ sung thông tin.
- Kiểm tra mượn/trả.
- Đã cập nhật giao dịch.
- Đang kiểm tra lỗi hệ thống.
- Kiểm tra tài khoản.
- Ghi nhận đề xuất sách.
- Thông báo đã xử lý.

Mẫu có placeholder:

```
{senderName}  
{ticketCode}
```

```
{librarianName}
```

Khi thủ thư chọn mẫu, frontend replace placeholder bằng dữ liệu ticket hiện tại.

Lợi ích:

- Phản hồi nhanh hơn.
- Giọng văn nhất quán.
- Giảm lỗi thiếu thông tin.
- Thủ thư mới vẫn xử lý được theo chuẩn.

Đây là một chi tiết nhỏ nhưng làm trang từ "dùng được" thành "vận hành được".

15. Những điểm chống lỗi dữ liệu

Hệ thống có nhiều lớp guard:

1. Ticket phải đăng nhập mới tạo được.
2. User không tự khai email để giả danh người khác.
3. User chỉ thấy ticket của mình.
4. Staff xem được toàn bộ, nhưng admin không được xử lý.
5. Ticket phải được assign trước khi thủ thư reply/resolve.
6. Chỉ assigned librarian được cập nhật ticket.
7. Ticket **RESOLVED**/**CLOSED** không cho chat tiếp theo luồng thường.
8. User chỉ được reopen khi ticket đã **RESOLVED** hoặc **CLOSED**.
9. User chỉ được feedback khi ticket đã **RESOLVED** hoặc **CLOSED**.
10. Internal note chỉ tác giả được sửa/xóa.

Nhờ vậy dữ liệu không bị loạn dù nhiều người cùng mở hệ thống.

16. Những giới hạn hiện tại

Hệ thống hiện chưa phải realtime thực sự.

Không có WebSocket hoặc Server-Sent Events cho contact tickets. Vì vậy:

- Nếu user gửi tin, thủ thư cần refresh hoặc chọn lại để thấy cập nhật.
- Nếu thủ thư trả lời, user nhận notification/email nhưng conversation không tự push ngay nếu đang mở sẵn.

Tuy nhiên đây là trade-off hợp lý cho LMS:

- Đơn giản hơn.
- Ít hạ tầng hơn.
- Dễ debug hơn.
- Không cần quản lý connection realtime.
- Với ticket hỗ trợ thư viện, độ trễ vài giây hoặc cần refresh thường chấp nhận được.

Nếu sau này cần realtime, có thể thêm:

- WebSocket topic theo **contactMessageId**.

- Server-Sent Events cho notification/comment mới.
- Polling nhẹ mỗi 15-30 giây khi đang mở thread.

Nhưng không nên thêm realtime quá sớm nếu vận hành hiện tại đã đủ.

17. Cách làm ra một trang tốt như vậy

Muốn làm ra một trang kiểu này, không bắt đầu từ UI. Phải bắt đầu từ workflow.

Các câu hỏi đúng:

1. Ai tạo yêu cầu?
2. Ai được thấy yêu cầu?
3. Ai được nhận xử lý?
4. Nếu hai người cùng nhận thì sao?
5. Người xử lý có quyền gì?
6. Người không phụ trách có được sửa không?
7. Khi xử lý xong, user còn quyền phản hồi không?
8. Nội dung nào user thấy, nội dung nào chỉ nội bộ thấy?
9. Khi có phản hồi, user được báo bằng cách nào?
10. Admin giám sát thế nào mà không phá quy trình?

Sau khi trả lời được workflow, mới thiết kế:

- Database schema.
- API contracts.
- State transitions.
- Permission checks.
- UI states.
- Notification.
- Error handling.

Trang `contact-tickets` tốt vì nó có đủ các lớp này, không dừng ở việc tạo form gửi message.

18. Bài học kiến trúc rút ra

Một hệ thống hỗ trợ tốt cần có ownership.

Trong Library74, ownership nằm ở `assigned_to_user_id`. Đây là cột nhỏ nhưng quyết định chất lượng vận hành. Nó biến một danh sách message lộn xộn thành một queue xử lý có trách nhiệm.

Một hệ thống chat tốt cần có thread.

Trong Library74, thread nằm ở `contact_message_comments`. Nó giúp mọi trao đổi cùng nằm trong một dòng thời gian, không bị tách giữa "nội dung ban đầu" và "phản hồi".

Một hệ thống vận hành tốt cần tách public communication và internal note.

Trong Library74, user-facing chat và staff-only notes là hai bảng khác nhau. Đây là thiết kế đúng vì không phải mọi thông tin xử lý nội bộ đều nên gửi cho user.

Một hệ thống nhiều người dùng tốt cần đưa critical section xuống backend/database.

Trong Library74, frontend có disabled state, nhưng backend mới là lớp quyết định. Câu `UPDATE ... WHERE assigned_to_user_id IS NULL OR assigned_to_user_id = :userId` là điểm giữ an toàn khi nhiều thủ thư thao tác cùng lúc.

19. Checklist khi bảo trì hoặc mở rộng

Khi sửa trang này, luôn kiểm tra:

- Có phá rule "chỉ assigned librarian được xử lý" không?
- Có cho admin vô tình update ticket không?
- Có để user xem ticket người khác không?
- Có tạo comment nhưng quên update `updated_at` không?
- Có gửi notification sai người không?
- Có cho chat tiếp trên ticket đã closed không?
- Có làm mất distinction giữa comment và internal note không?
- Có làm response template gửi thẳng mà không cho thủ thư chỉnh lại không?
- Có xử lý lỗi `409 CONFLICT` rõ ràng trên UI không?

20. Kết luận

`contact-tickets` là một ví dụ tốt về cách biến một chức năng nhỏ thành một workflow đáng tin cậy.

Nó có:

- Ticket lifecycle rõ ràng.
- User inbox riêng.
- Librarian inbox riêng.
- Admin monitoring read-only.
- Thread chat theo ticket.
- Internal notes cho staff.
- Assignment để chống nhiều người xử lý cùng lúc.
- Notification và email để kéo user quay lại.
- Feedback loop sau khi resolve.

Phần quan trọng nhất không phải giao diện đẹp, mà là logic ownership và state transition đúng. UI tốt là kết quả của backend model đúng: khi ticket có owner, status, comments, notes, feedback và notification rõ ràng, frontend có thể render thành một trải nghiệm rất mượt và rất dễ vận hành.